



Excel Practicals

Complete Study Guide

Pivot Tables • Goal Seek • Scenario Manager • Solver

Practical 1: Import Data + Pivot Tables + Charts

Step 1 — Import Data from SQL Server

1. Open Excel
2. Go to Data → Get Data → Legacy Wizards → From SQL Server
3. Enter the Server Name (copy from SQL Server properties)
4. Click Next
5. Select Multiple files / Select related tables
6. Click Next → Finish

Step 2 — Create Pivot Table

7. Go to Insert → Pivot Table
8. Choose Use External Connection / Choose Connection

Case (a) — Quantity by Product (Pie Chart)

Pivot Area	Field
Filters	SalesDateKey
Rows	ProductName
Values	Quantity

Chart: Insert → Charts → Pie Chart

Case (b) — Sales Total Cost by Person & Product

Pivot Area	Field
Columns	SalesPersonName
Rows	ProductName
Values	Sales Total Cost

Case (c) — Sales Total Cost by Customer & Product

Pivot Area	Field
Columns	CustomerName
Rows	ProductName
Values	Sales Total Cost

Practical 2: Financial Analysis Tools

Case (a) — Loan PMT + Goal Seek

Given Data

Parameter	Cell	Value
Annual Interest Rate	B1	6%
Years	B2	30
Loan Amount	B3	300,000
PMT (Formula)	B4	=PMT(B1/12, B2*12, B3)

PMT Formula

```
=PMT(B1/12 , B2*12 , B3)
```

Goal Seek Steps — Target Monthly Payment: \$1,500

9. Go to Data → What-If Analysis → Goal Seek
10. Set Cell → B4 (the PMT cell)
11. To Value → 1500
12. By Changing Cell → B3 (Loan Amount)
13. Click OK — Excel finds the loan amount that gives \$1,500/month

Case (b) — Scenario Manager (Profit Analysis)

Input Variables

Variable	Abbreviation	Base Value
Revenue	–	500,000
Cost of Goods Sold	COGS	300,000
Distribution Expense	DE	100,000
Marketing Expense	ME	50,000
R&D Expense	RD	20,000
Interest Expense	IE	10,000
Tax Rate	–	25%

Calculation Formulas

Metric	Short	Formula
Gross Profit	GP	Revenue – COGS
Operating Profit	OP	GP – SUM(DE, ME, RD)
Profit Before Tax	PBT	OP – IE
Net Profit	NP	PBT – (PBT × Tax Rate)

Scenario Manager Steps

14. Go to Data → What-If Analysis → Scenario Manager
15. Click Add → enter Scenario Name (e.g. Worst Case, Best Case, Base Case)
16. In Changing Cells, select value cells of: Revenue, COGS, DE, ME, RD, IE, Tax Rate
17. Click OK → enter values for each scenario
18. Click Summary
19. In Result Cells, select: Gross Profit, Operating Profit, Profit Before Tax, Net Profit
20. Click OK — Excel generates a summary worksheet

Case (c) — Solver: Product Mix Optimization (Chair & Table)

Step 1: Enter Data in Excel

Machine	Chair	Table	Available Time
M1	3	3	36
M2	5	2	50

M3	2	6	60
Decision Variables	0	0	—
Contribution (Profit)	20	30	Maximize

Step 2: Calculate Time Used (SUMPRODUCT Formula)

In the Time Used (Profit) cell for M1, enter:

```
=SUMPRODUCT(DV_Range , M1_Chair_Table_Range)
```

Important: After selecting DV_Range, press F4 to make it an absolute reference (\$). Then drag the formula down for M2 and M3.

Step 3: Calculate Total Profit

```
=SUMPRODUCT(DV_Range , Contribution_Range)
```

Press F4 after selecting DV_Range to lock the reference.

Step 4: Enable Solver Add-in

21. File → Options → Add-ins
22. At the bottom, select Excel Add-ins → Go
23. Tick Solver Add-in → OK

Steps 5–10: Configure and Run Solver

#	Solver Field	Setting
5	Open Solver	Data → Solver
6	Set Objective	Select Total Profit Cell → Max
7	By Changing Variables	Select Decision Variable cells (Chair & Table)
8	Constraint 1	M1 Time Used ≤ 36
8	Constraint 2	M2 Time Used ≤ 50
8	Constraint 3	M3 Time Used ≤ 60
8	Constraint 4	DV Cells ≥ 0 (non-negativity)
9	Solving Method	Simplex LP
10	Solve	Click Solve → Keep Solver Solution → OK

Quick Reference Summary

Tool	Purpose	Location in Excel
Pivot Table	Summarise & cross-tabulate data	Insert → Pivot Table
Goal Seek	Find input for a target output	Data → What-If → Goal Seek
Scenario Manager	Compare multiple what-if scenarios	Data → What-If → Scenario Manager
Solver	Optimise with constraints (LP)	Data → Solver (Add-in required)

R & Python Practicals

Practical Exam Reference Guide · Practicals 3 – 8

Classification · Clustering · Regression · Logistic Regression · Data Analysis · Visualization

Practical 3 — Classification Algorithm in R

Part 1 — Time Series: Annual Rainfall

Create an R time series object for annual rainfall (Jan 2012, 12 months) and plot it.

Steps

Open RStudio → File → New → R Script → paste code → Run

```
# Step 1: Enter rainfall data (mm) for 12 months from Jan 2012
rainfall <- c(799, 1174.8, 865.1, 1334.6, 635.4, 918.5,
             685.5, 998.6, 784.2, 985, 882.8, 1071)

# Step 2: Create time series object
rainfall.timeseries <- ts(rainfall, start = c(2012, 1), frequency = 12)

# Step 3: Print the time series
print(rainfall.timeseries)

# Step 4: Save & plot
png(file = "rainfall.png")
plot(rainfall.timeseries)
dev.off()
```

OUTPUT: Time series printed in Console + rainfall.png saved. Use Plots panel → `plot(rainfall.timeseries)` if graph not visible.

Part 2 — Decision Tree (Party Package)

Use the built-in `readingSkills` dataset and the `cree()` function to build a classification decision tree.

```
# Step 1: Install and load the party package
install.packages("party")
library(party)

# Step 2: Load dataset (first 105 rows)
data(readingSkills)
```

```
input.dat <- readingSkills[c(1:105), ]

# Step 3: Build conditional inference tree
output.tree <- ctree(
  nativeSpeaker ~ age + shoeSize + score,
  data = input.dat
)

# Step 4: Save & plot
png(file = "decision_tree.png")
plot(output.tree)
dev.off()
```

OUTPUT: Decision tree plot saved as decision_tree.png. Use Plots panel → plot(output.tree) if graph not visible.

Practical 4 — Data Clustering

Part A — K-Means Clustering in R (iris dataset)

Use the built-in iris dataset. Remove Species column, apply K-Means with 3 clusters, then plot.

Steps

Open RStudio → paste directly in Console

```
# Step 1: Copy iris dataset and remove Species column
newiris <- iris
newiris$Species <- NULL

# Step 2: Apply K-Means with 3 clusters
kc <- kmeans(newiris, centers = 3)
kc

# Step 3: Compare clusters with actual species
table(iris$Species, kc$cluster)

# Step 4: Plot clusters
plot(newiris[c("Sepal.Length", "Sepal.Width")],
     col = kc$cluster,
     main = "K-means Clustering")

# Step 5: Add cluster centres
points(kc$centers[, c("Sepal.Length", "Sepal.Width")],
       col = 1:3, pch = 8, cex = 2)
```

OUTPUT: Cluster table in Console + scatter plot in Plots panel.

Part B — DBSCAN Clustering in Python

Density-Based Spatial Clustering of Applications with Noise using scikit-learn.

Setup

Open Spyder → run in Console: `pip install numpy matplotlib scikit-learn`

```
# Paste in a new .py file (e.g. temp.py)
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN

# Step 1: Generate synthetic dataset
X, y = make_blobs(n_samples=300, centers=4,
                  cluster_std=0.50, random_state=0)

# Step 2: Apply DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X)

# Step 3: Visualise clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='plasma', s=50)
plt.title("DBSCAN Clustering Example")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

OUTPUT: Scatter plot showing DBSCAN clusters (noise points shown as -1).

Part C — K-Means Clustering in Python

Apply K-Means with 4 clusters on synthetic data and visualise centroids.

Setup

Open Spyder → run in Console: `pip install numpy matplotlib scikit-learn`

```
# Paste in a new .py file (e.g. temp.py)
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Step 1: Generate synthetic dataset
X, y = make_blobs(n_samples=300, centers=4,
                  cluster_std=0.60, random_state=0)
```



```
# Step 2: Apply K-Means
kmeans = KMeans(n_clusters=4, random_state=0)
kmeans.fit(X)

# Step 3: Get labels and centroids
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Step 4: Visualise
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1],
            c='red', s=200, alpha=0.7, marker='X')
plt.title("K-Means Clustering Example")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

OUTPUT: Scatter plot with 4 colour-coded clusters and red X markers at centroids.

Practical 5 — Linear Regression in R

Predict weight (kg) from height (cm) using the `lm()` function. Run each section as a separate R Script.

Step 1 — Build Model & Get Coefficients

```
x <- c(151, 174, 138, 186, 128, 136, 179, 163, 152, 131) # Height
y <- c(63, 81, 56, 91, 47, 57, 76, 72, 62, 48) # Weight

relation <- lm(y ~ x)
print(relation)
```

OUTPUT

```
Coefficients:
(Intercept)      x
-38.4551      0.6746
```

Step 2 — Full Model Summary

```
x <- c(151, 174, 138, 186, 128, 136, 179, 163, 152, 131)
y <- c(63, 81, 56, 91, 47, 57, 76, 72, 62, 48)

relation <- lm(y ~ x)
print(summary(relation))
```

OUTPUT: $R^2 = 0.9548$ | Adjusted $R^2 = 0.9491$ | p-value = 1.16e-06 (strong fit)

Step 3 — Predict Weight for Height = 170 cm

```
x <- c(151, 174, 138, 186, 128, 136, 179, 163, 152, 131)
y <- c(63, 81, 56, 91, 47, 57, 76, 72, 62, 48)

relation <- lm(y ~ x)
a <- data.frame(x = 170)
result <- predict(relation, a)
print(result)
```

OUTPUT: Predicted weight for height 170 cm → 76.22869 kg

Step 4 — Plot Regression Graph

```
x <- c(151, 174, 138, 186, 128, 136, 179, 163, 152, 131)
y <- c(63, 81, 56, 91, 47, 57, 76, 72, 62, 48)

relation <- lm(y ~ x)

# Save to PNG
png(file = "linearregression.png")
plot(y, x,
     col = "blue",
     main = "Height & Weight Regression",
     abline(lm(x ~ y)),
     cex = 1.3,
     pch = 16,
     xlab = "Weight in Kg",
     ylab = "Height in cm")
dev.off()

# To view in Plots panel – run this line separately:
plot(y, x, col="blue", main="Height & Weight Regression",
     abline(lm(x~y)), cex=1.3, pch=16,
     xlab="Weight in Kg", ylab="Height in cm")
```

OUTPUT: Scatter plot with regression line saved as linearregression.png. View in Plots panel.

Practical 6 — Logistic Regression in R

Predict the vs (engine type) variable in the mtcars dataset using weight (wt) and displacement (disp).

Step 1 — Import Dataset

```
install.packages("dplyr")
library(dplyr)
```

```
head(mtcars) # View first 6 rows
```

Step 2 — Split into Train / Test Sets

```
install.packages("caTools")
library(caTools)

split <- sample.split(mtcars, SplitRatio = 0.8)
train_reg <- subset(mtcars, split == "TRUE")
test_reg <- subset(mtcars, split == "FALSE")
```

Step 3 — Build Logistic Regression Model

```
logistic_model <- glm(vs ~ wt + disp,
                      data = train_reg,
                      family = "binomial")

logistic_model # Print model
summary(logistic_model) # Full summary with significance
```

OUTPUT: Model coefficients, AIC score, and significance codes printed in Console.

Linear vs Logistic Regression

Linear: continuous output (e.g. weight)
Logistic: binary output (0 or 1) using sigmoid

glm() key arguments

family = "binomial" -> logistic
vs ~ wt + disp -> formula

Practical 7 — Write a Python program to read data from a CSV file, perform simple data analysis, and generate basic insights. (Use Pandas is a Python library)

Requirement

A CSV file named Sales_Data.csv must be in the working directory.

Program 1 — Revenue & Sales Insights + Charts

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv(r"Sales_Data.csv")

# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'])
```

```

# Basic exploration
print(df.head())
print("Dataset Info")          ; print(df.info())
print("Statistical Summary") ; print(df.describe())

# Revenue analysis
total_revenue = df['TotalPrice'].sum()
print("Total Revenue Generated:", total_revenue)

region_sales = df.groupby('Region')['TotalPrice'].sum()
print("Sales by Region")      ; print(region_sales)

popular_product = df.groupby('Product')['Quantity'].sum().idxmax()
print("Most Selling Product:", popular_product)

# Yearly sales line chart
df['Year'] = df['Date'].dt.year
yearly_sales = df.groupby('Year')['TotalPrice'].sum()
plt.plot(yearly_sales.index, yearly_sales.values, marker='o')
plt.title("Yearly Sales Performance")
plt.xlabel("Year")
plt.ylabel("Total Sales ($)")
plt.grid(True)
plt.show()

# Revenue by product bar chart
category_data = df.groupby('Product')['TotalPrice'].sum().sort_values()
plt.figure(figsize=(10, 6))
category_data.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title("Total Revenue by Product Category")
plt.xlabel("Product")
plt.ylabel("Total Sales ($)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

OUTPUT: Console: total revenue, regional sales, most-selling product. Plots: yearly trend + bar chart by product.

Program 2 — Dataset Summary & Basic Statistics

```

import pandas as pd

file_path = "Sales_Data.csv"
df = pd.read_csv(file_path)

print("\n First 5 rows of the dataset:")
print(df.head())

print("\n Dataset Summary:")
print(df.info())

```

```

print("\n Basic Statistics:")
print(df.describe())

print("\n Missing Values in Each Column:")
print(df.isnull().sum())

print("\n Column Names:")
print(df.columns)

# Conditional analysis (runs only if column exists)
if 'Salary' in df.columns:
    print(f"\n Average Salary: {df['Salary'].mean():.2f}")

if 'Age' in df.columns:
    print(f"\n Youngest Person's Age: {df['Age'].min()}")
    print(f"\n Oldest Person's Age: {df['Age'].max()}")

if 'Gender' in df.columns:
    print("\n Gender Distribution:")
    print(df['Gender'].value_counts())

print("\n Data Analysis Completed Successfully!")

```

OUTPUT: Console: shape, dtypes, missing values, stats, and conditional column-specific insights.

Practical 8 — Data Visualization with Python

Generate sample sales data and create 4 different visualizations using Matplotlib and Seaborn.

Setup

Terminal / Spyder Console: `pip install pandas numpy matplotlib seaborn`

IDE

Open Spyder → paste entire code block into a new .py file → Run

Full Program — 4 Visualizations

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# — Generate Sample Sales Data —————
dates      = pd.date_range(start='2023-01-01', periods=100, freq='D')
categories = ['Electronics', 'Clothing', 'Groceries', 'Furniture']

```

```

data = {
    'Date':      np.random.choice(dates, 200),
    'Category':  np.random.choice(categories, 200),
    'Sales_Amount': np.random.randint(100, 1000, 200),
    'Units_Sold': np.random.randint(1, 20, 200)
}
df = pd.DataFrame(data)
df['Date'] = pd.to_datetime(df['Date'])

# Summary statistics
print(df.describe())

# — Plot 1: Sales Trend Over Time (Line Chart) —————
plt.figure(figsize=(12, 6))
sns.lineplot(
    data=df.groupby('Date')['Sales_Amount'].sum().reset_index(),
    x='Date', y='Sales_Amount'
)
plt.title('Sales Trend Over Time')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.xticks(rotation=45)
plt.show()

# — Plot 2: Total Sales by Category (Bar Chart) —————
plt.figure(figsize=(10, 5))
sns.barplot(
    data=df.groupby('Category', as_index=False)['Sales_Amount'].sum(),
    x='Category', y='Sales_Amount', palette='viridis'
)
plt.title('Total Sales by Category')
plt.xlabel('Category')
plt.ylabel('Sales Amount')
plt.show()

# — Plot 3: Distribution of Sales Amounts (Histogram) —————
plt.figure(figsize=(10, 5))
sns.histplot(df['Sales_Amount'], bins=20, kde=True)
plt.title('Distribution of Sales Amounts')
plt.xlabel('Sales Amount')
plt.ylabel('Frequency')
plt.show()

# — Plot 4: Sales Amount vs Units Sold (Scatter Plot) —————
plt.figure(figsize=(10, 5))
sns.scatterplot(
    data=df, x='Units_Sold', y='Sales_Amount',
    hue='Category', palette='coolwarm'
)
plt.title('Sales Amount vs Units Sold')
plt.xlabel('Units Sold')
plt.ylabel('Sales Amount')
plt.show()

```

#	Chart Type	What it Shows
1	Line Chart	Daily sales trend over time (Jan–Apr 2023)
2	Bar Chart	Total revenue comparison across 4 product categories
3	Histogram + KDE	Frequency distribution of individual sale amounts
4	Scatter Plot	Relationship between units sold and sales amount per category

Quick Reference — All Practicals

Prac	Language	Topic	Key Function	Output
3a	R	Time Series	ts() + plot()	Line plot
3b	R	Decision Tree	ctree() (party)	Tree diagram
4a	R	K-Means	kmeans()	Scatter + table
4b	Python	DBSCAN	DBSCAN()	Scatter plot
4c	Python	K-Means	KMeans()	Scatter + centroids
5	R	Linear Regression	lm() + predict()	Plot + coefficients
6	R	Logistic Regression	glm(family='binomial')	Model summary
7	Python	Data Analysis	pandas (read_csv)	Console + 2 plots
8	Python	Visualization	seaborn / matplotlib	4 plots